SEAT NO. _____

NED UNIVERSITY OF ENGINEERING & TECHNOLOGY

FIRST YEAR (Computer Science)

SPRING SEMESTER EXAMINATIONS 2025

Time : 3 Hours | Batch 2024 | Dated : 27-MAY-25 | Max Marks : 60

Object Oriented Programming - CT-260

1. Review following given codes and **identify** any errors that might occur. If there are no errors, then write output of the programs. [CLO-01, Marks 10]

a.

```
template<class T, class U, class V=double>
class A {
public:
   T x;
   U y;
   V z;
   static int count;
};
int main(){
  A<int, int> a;
  A<double, double> b;
  cout << sizeof(a.x) << endl;
  cout << sizeof(b.z) << endl;
  return 0;
}
```

b.

```
class Shape {
public:
  void draw() {
    cout << "Drawing shape" << endl; }
};
class Circle : public Shape {
public:
  void draw() {
    cout << "Drawing circle" << endl; }
};
void render(Shape* s) {
  s->draw();  }
int main(){
  Shape *obj= new Circle;
  render(obj);  }
```

c.

```
class Base{
public:  static int count;
Base() {count++;}
virtual ~Base() {count--;}
static void printCount(){
cout << "Count: " << count << endl;  }
};
class Derived : public Base {
public:
 Derived() {count++;}
~Derived() {count--;} };
int Base::count = 0;
int main() {
 Base::printCount();
 Base obj1;
 Derived* obj2 = new Derived;
 obj2->printCount();
 delete obj2;
 obj1.printCount();  }
```

d.

```
class Book {
private: string* title; bool borrowed;
public:
   Book(string t) : title(new string(t)),
borrowed(false) { }
Book(const Book& other){
  title = new string(*other.title);
 }
void operator=(Book other) {
*title = *other.title;
}
void status() {
    cout << *title << ": " << borrowed;  }
};
int main() {
  Book b1("Book1");
  Book b2 = b1;
  b2 = b1;
  b2.status();  }
```

True

2. Answer the following short questions. [CLO-01, Marks 10]
   a. **Define** with the help of suitable examples, IS-A and HAS-A relationships.
   b. **Differentiate** between function templates and overloaded functions?
   c. **Explain** how read and write methods of fstream object can be used to read or write objects in a binary file with the help of a sample code example.
   d. **Define** friend functions. **Which** pillar of object oriented programming do they violate and **why?**

3. **Draw** UML Class Diagram for the following scenario. [CLO-01, Marks 10]

The university is planning to develop a Course Management System that will help streamline the administration of academic courses, student enrollments, and faculty responsibilities. This system will be used by three types of users: students, instructors, and administrators. All users will have some common characteristics such as a unique user ID, name, and email address. However, each type of user will also have specific attributes and behaviors relevant to their role in the system.

Students will use the system to register for academic courses. Each student can enroll in multiple courses but is limited to a maximum of five active course enrollments at any time. A student can also drop a course they are enrolled in and should be able to view a list of all their current courses. Once a course is completed, the instructor will assign a final grade to the student, which the student can later access through the system.

Instructors are responsible for managing the courses they teach. Each course is created and maintained by an instructor and includes details such as a unique course code, title, description, number of credits, and the maximum number of students that can be enrolled. Instructors can access the list of students registered for each of their courses, post grades, and schedule lectures by specifying time slots and meeting days.

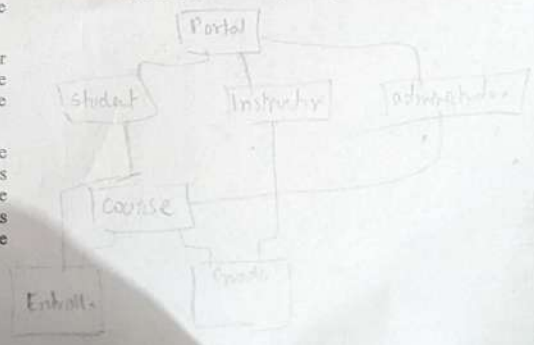
Administrators oversee the overall functioning of the system. They are not directly involved in teaching or learning but have access to analytical and monitoring tools. They can generate summary reports that include statistics such as how many students are enrolled in each course and how many courses each instructor is teaching. Administrators can also approve or deactivate courses and update user access when necessary.

The system should support intuitive operations between objects using operator overloading. For example, adding a student to a course should be as simple as using the '+' operator, where the expression course + student results in the student being added to the course roster, provided the course is not full and prerequisites are satisfied.

Your task is to represent the system using a UML class diagram that captures the core structure and relationships of its components. Focus on identifying appropriate classes, their key attributes and methods, and the associations between them. Your diagram should clearly convey inheritance hierarchies and any relevant abstractions. Consider how object-oriented principles such as encapsulation, inheritance, and polymorphism are reflected in your model, and think about where operator overloading might be applicable.

4. A medical store sells both prescription and over-the counter (OTC) medicines. Some medicines, like certain pain relievers, can fall into both categories under specific conditions. **Construct** a management system for this medical store, by following the rules given below:
   - create an abstract base class Medicine that contains protected attributes like name, price, and a pure virtual function displayDetails().
   - derive two classes from Medicine:
     - PrescriptionMedicine with additional data members like doctorName, patientName, and prescriptionDate.
     - OTCMedicine with attributes like maxDailyDosage and ageRestriction.
   - Implement a HybridMedicine class (e.g., a medicine that can be both OTC and prescription) that inherits from both PrescriptionMedicine and OTCMedicine.
   - Create a list of Medicine* and dynamically call displayDetails() for different medicine types at runtime.

Apply the OOP concepts required for solving this problem. [CLO-02, Marks 10]

5. **Construct** a template function searchElement that takes three inputs: an array, the size of the array, and the element to search for. The function should search for the element in the array (which can be of any type) and return its index. If the element is not found, the function should throw an exception. Test the template function by calling it with arrays of type float and char. Also, handle the exception appropriately in the main function. [CLO-02, Marks 10]

6. **Construct** a system for an online store to manage shopping carts. Create an Item class with dynamic attributes (name, ID, price, and a boolean for in-stock status), constructor, copy constructor, destructor, and overloaded == (compare by ID) and < (compare by price). Create a Cart class to store items in a vector, with methods to add, remove, and find items by ID. Ensure proper memory management and edge case handling. [CLO-02, Marks 10]